

LLM 后训练之 RL / RLVR

从 PPO / GRPO 到可验证推理、长 CoT 稳定化与生产级 Rollout 系统

深度研究与工程手册 · 2026-08

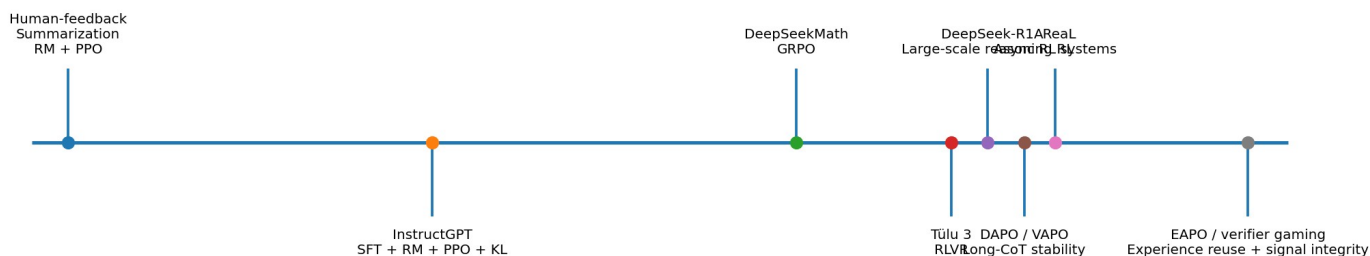


图 1 LLM 后训练 RL 的公开演进：从 RM+PPO 到 RLVR、Long-CoT 与 Experience Reuse

核心判断

LLM RL 的本质不是“把 reward 最大化”，而是：在当前 policy 附近持续制造可学习、可信、足够多样且不过时的经验，再用受约束的 policy gradient 将这些经验写入模型。现代 reasoning RL 的难点已经从单一算法，转移到 Rollout、Verifier、Advantage、Sampling、Stability 与 Systems 的联合设计。

阅读地图

层	核心问题	本手册对应章节
1. Objective	为什么要用 RL?	1-3: 第一性原理、与 SFT/DPO 边界
2. Experience	模型看到什么经验?	4-6: MDP、rollout、online/off-policy
3. Signal	如何知道轨迹好坏?	7-10: reward、verifier、advantage、KL
4. Optimization	怎样把 reward 写进 policy?	11-16: REINFORCE/RLOO/PPO/GRPO/DAPO/VAPO
5. Stability	为什么会崩?	17-21: entropy collapse、length bias、reward hacking、credit assignment
6. Systems	怎样训得动?	22-25: rollout engine、同步/异步、reshard、staleness
7. Production	如何成为持续迭代系统?	26-30: recipe、eval、failure taxonomy、maturity、checklist

建议阅读顺序

如果你关注 know-why: 先读 1 → 3 → 9 → 16 → 21; 如果你关注 know-how: 先读 6 → 12 → 14 → 17 → 22 → 26; 如果你在搭 reasoning / coding agent RL 系统: 重点读 8、14、17、20、22-29。

1. RL 在 LLM 后训练中的第一性角色

SFT 给模型“示范”; Preference Optimization 给模型“相对排序”; RL 给模型的是“交互后得到的标量或结构化反馈”。它允许 policy 自己采样候选行为, 并通过 reward 的差异重新分配这些行为的概率质量。InstructGPT 使用 RM+PPO 将人类偏好写回 policy; DeepSeek-R1-Zero 则展示了可验证 reasoning task 上纯 RL 能诱发自验证、反思和更长推理等行为。[1][5]

阶段	监督信号	是否需要当前 Policy Rollout	核心能力
SFT	Demonstration	否	模仿高质量行为, 建立 behavior prior
DPO/Preference	chosen > rejected	通常否/弱	偏好边界与风格/行为校准
RLHF	Learned reward	是	在线探索并优化偏好 reward
RLVR	Rule / Test / Exact verifier	是	围绕可验证目标进行能力搜索
Agent RL	Environment return	是	长轨迹决策、工具调用、恢复与计划

Know-Why

RL 最大的独特性不是“还有一个 loss”, 而是训练数据分布由当前 policy 参与生成。换句话说, 数据生成器和被训练对象是同一个动态系统。

2. 从语言生成到 MDP : LLM RL 的建模方式

一种常见建模把 prompt/context 看作初始状态, 把每个 token 看作 action; 序列生成结束后获得 trajectory-level reward。另一种更贴近 LLM 对齐实践的视角, 是把完整 response 当作单个宏动作, 以 REINFORCE/RLOO 直接优化整条生成。RLOO 的研究显示, 在其 RLHF 设置里, 完整序列级建模可以比逐 token value-based PPO 更简单、更有效。[8]

元素	Token-level MDP	Sequence-level / Bandit-like 视角
State	prompt + partial completion	prompt
Action	next token	entire completion
Reward	可稀疏到 EOS	response-level reward
Baseline	critic / GAE	group / leave-one-out baseline
优点	可做细粒度 credit assignment	实现简单、减少 value model
风险	value bias、长序列复杂	中间步骤错误不容易定位

数学上最基本的 policy gradient 形式可写为：

$$\nabla_{\theta} J(\theta) = E[(R - b) \nabla_{\theta} \log \pi_{\theta}(y|x)]$$

其中 baseline b 不改变期望梯度方向，但可以显著降低方差。PPO 使用 value function/GAE；GRPO 使用同一 prompt 内的 group reward；RLOO 使用其余 samples 的 leave-one-out 平均作为 baseline。[2][4][8]

3. RLHF 与 RLVR：两个不同的工程世界

维度	经典 RLHF	RLVR / Reasoning RL
Reward 来源	Human preference → Reward Model	Exact match / unit test / compiler / theorem checker / environment
主要目标	helpfulness / harmlessness / style	math / code / formal / task success
Reward 风险	proxy overoptimization / RM hacking	verifier loophole / parser bug / extensional shortcut
KL 角色	通常强：限制离 SFT policy 太远	可弱化甚至去除，取决于任务
探索需求	中等	高；需要多个长 CoT candidates
关键瓶颈	RM quality + PPO stability	rollout cost + verifier quality + entropy + long-CoT stability

DAPO 明确区分了 reasoning RL 与传统 RLHF：其长 CoT 数学训练直接去掉 KL 项并使用 rule-based accuracy reward，目标是释放 base model 的 reasoning 分布，而不是维持对 SFT reference 的强邻域约束。[6] 这不意味着“所有 RL 都应该去 KL”；它意味着 KL 是目标相关的控制器，而不是教条。

4. Experience 是 RL 的真正“训练数据”

在 online RL 中，一条训练样本不是静态 (prompt, answer)，而是包含 policy version、采样配置、生成 token、old logprob、reward、verifier trace、termination reason 等信息的 trajectory。没有这些字段，后续 importance ratio、staleness control、debug 和 reward audit 都会失真。

字段	为什么必须保留
prompt_id / task_id	追踪任务分布与 hard/easy 演化
policy_version	判断 on-policy / stale degree
response tokens	训练主体
old_logprobs	PPO/GRPO importance ratio
reward components	区分 correctness / format / length / cost
verifier_version	防止 reward 规则改变后不可追溯
stop_reason	区分正常结束、截断、工具错误
token_count / latency	算 compute-normalized learning efficiency

核心数据结构

RL 的 Dataset 更准确地叫 Experience Buffer / Trajectory Store。生产系统应该把它当一等数据产品管理，而不是临时 tensor。

5. Rollout：探索空间由谁决定？

RL 能不能学到新策略，先取决于 rollout 有没有采到这些策略。temperature、top-p、group size、max tokens、prompt mix 与当前 policy entropy 共同决定搜索空间。DeepSeek-R1/GRPO、DAPO 等路线都依赖同一 prompt 多样本 rollout；如果 group 内候选几乎相同，relative advantage 失去意义。[5][6]

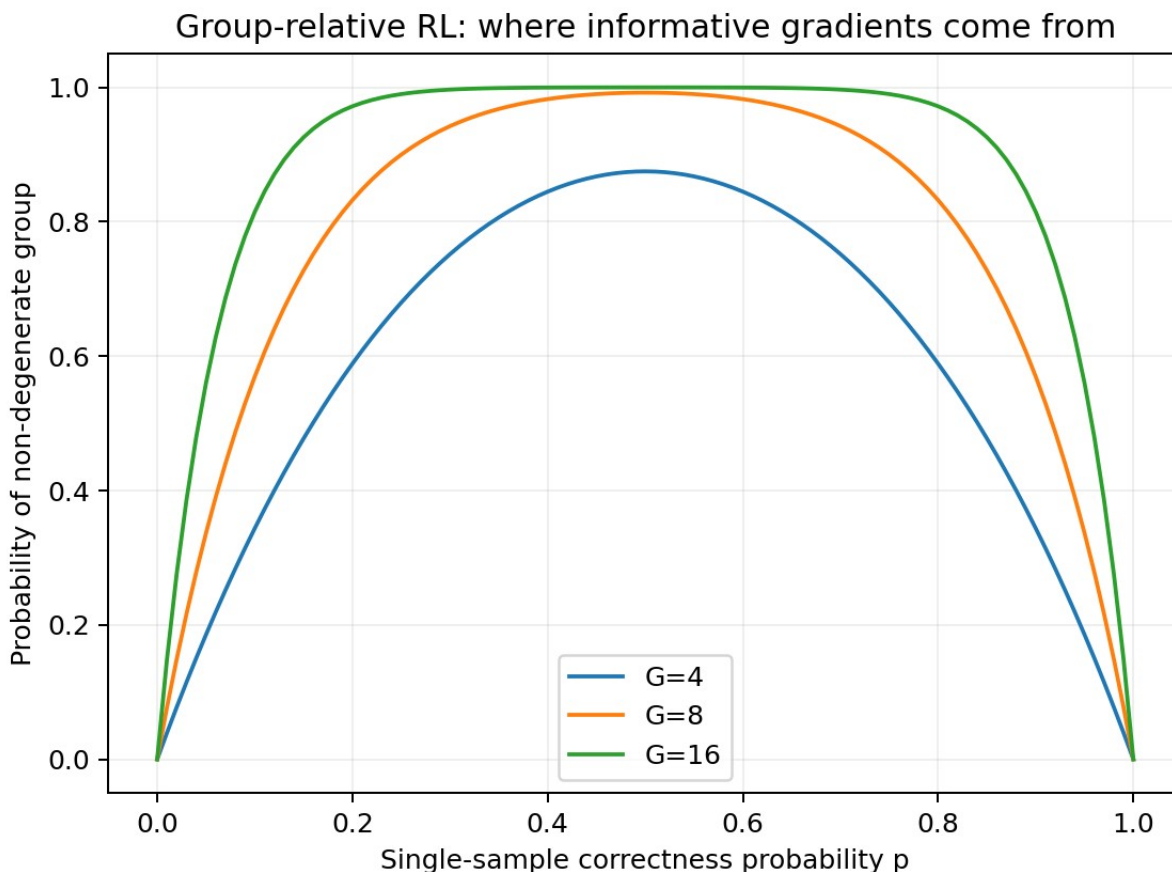


图 2 Group-relative RL 的有效信号概率。G 越大，越容易在中等难度 prompt 上同时采到正/负样本。

对 binary reward，若单样本答对概率为 p ，group size 为 G ，则“既有正确又有错误”的非退化 group 概率为：

$$P(\text{informative group}) = 1 - p^G - (1-p)^G$$

这解释了为什么 group-relative RL 天生偏爱“可学习边界”：太简单 ($p \approx 1$) 和太难 ($p \approx 0$) 的任务都难产生梯度。DAPO 的 Dynamic Sampling 正是显式过滤全对/全错 group，持续补采样直到 batch 中保留有有效梯度的 prompt。[6]

6. On-policy、Near-policy、Off-policy 与 Experience Freshness

RL 的一个根本约束是：更新 policy 时使用的 experience 是由哪个 policy 生成的？同步 RL 强调每一批数据来自当前/最近 policy；异步系统提升吞吐，但会引入 staleness。AReaL 通过完全解耦 generation 与 training 获得最高 $2.57\times$ 的训练加速，同时需要显式控制数据陈旧度并使用 staleness-aware PPO。[13]

类型	优点	风险	适用
On-policy	梯度最干净	生成昂贵、吞吐受慢样本拖累	稳定研究与高风险训练
Near-policy	兼顾新鲜度与吞吐	需定义最大 policy lag	大规模同步/半异步 RL
Off-policy replay	复用历史经验、降低 rollout cost	importance mismatch、策略行为过时	需要严格 correction / experience selection
Experience prior	利用旧 RL policy 引导新 rollout	可能把旧偏见写回新 policy	2026 EAPO 类方法探索

2026 的 EAPO 进一步提出：历史经验不应简单作为固定 trajectory 重放，而应以 policy-adaptive 的 action-level prior 注入新 rollout，并配合 importance sampling，试图解决经验复用与 policy mismatch 的矛盾。[16]

7. Reward：你实际上在优化什么？

Reward 是 RL 的“目标接口”。从工程角度，它至少有五类：learned reward model、rule-based reward、verifiable outcome reward、process reward、environment return。真正危险的不是 reward 低，而是 reward 与真实目标之间存在可被 policy 利用的空隙。Reward Model Overoptimization 的研究系统展示了：对 imperfect proxy 持续优化，proxy reward 上升的同时 gold reward 可能下降。[3]

Reward 形式	例子	优点	主要风险
Learned RM	preference score	覆盖开放任务	Goodhart / model exploitation
Rule	格式、规则组合	稳定、可解释	规则遗漏与边界漏洞
Verifier	答案等价、unit test	客观、便宜	false positive/false negative
PRM	步骤级正确性	credit 更密集	标注/模型成本高、step evaluator 也可被 hack
Environment	agent task success	最接近真实目标	成本、不可逆 side effects、长 horizon

8. Verifier 不是“绝对真理”：RLVR 的新风险

RLVR 常被误解为“有 verifier 就没有 reward hacking”。事实上 verifier 只验证它编码的属性。2026 的研究展示，在归纳推理任务中，RLVR policy 会学会枚举实例级标签以通过 extensional verifier，而不是学习应有的抽象规则；加入同构扰动验证后，这种 shortcut 才暴露出来。[17]

Know-How

Verifier 设计要问三个问题：① false positive 会奖励什么捷径？② false negative 会压制哪些正确但非 canonical 的答案？③ policy 能否直接观察/推断 verifier 的漏洞？高价值 RLVR 需要 verifier diversity、hidden tests、perturbation tests 与独立 eval。

9. Advantage：把“得分”变成“更新方向”

Reward 本身不直接说明“这个 sample 比预期好多少”。Advantage 是 reward 相对 baseline 的残差。不同 RL 算法最大的差异之一，就是 baseline/advantage 怎么估计。

方法	Baseline / Advantage	需要 Critic?	典型特征
PPO + GAE	V(s), TD residual, GAE	是	通用、细粒度，但 critic 复杂
REINFORCE	moving average / scalar baseline	否	简单但方差较大
RLOO	同 prompt 其他 samples 的 leave-one-out reward	否	多样本方差降低
GRPO	group mean/std normalized reward	否	适合多候选可验证任务
DAPO	group advantage + dynamic group selection	否	强化有效 group 与 long-CoT 稳定

10. KL：约束、奖励塑形还是不需要？

经典 RLHF 常把 KL penalty 加到 reward: $R_{\text{total}} = R_{\text{RM}} - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$ 。其目的既是防止过度偏离 SFT policy，也是减缓 reward model overoptimization。InstructGPT 和早期 RLHF recipe 都强依赖这一结构。[1][3]

但 DAPO 在 long-CoT reasoning 中明确移除了 KL，理由是此时希望 policy 能显著离开初始分布；DeepSeek-R1 的 GRPO 表述则仍保留 reference KL。[5][6] 因此正确工程问题是：“当前任务中，reference policy 代表的是必须保留的行为先验，还是阻碍探索的旧分布？”

β 太大	β 合理	β 太小/无 KL
reward 学不动；reasoning length/策略难扩展	在能力提升与行为稳定之间取平衡	更强探索，但更容易 policy drift、reward hacking、语言退化

11. PPO：为什么它成为经典 RLHF 默认算法？

PPO 通过 importance ratio 的 clipped surrogate objective 限制单次 policy 更新幅度，同时可对同一批 rollout 做多轮 minibatch 更新。[2] 在 LLM RLHF 中通常还会联合训练 value model、加入 KL reward、whiten rewards/advantages、处理 EOS/length 等大量实现细节。Open reproduction 表明，超过 20 个实现细节足以显著影响结果。[9]

PPO clipped objective（简化）：

$$L = E[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)]$$

$$r_t = \pi_{\theta}(a_t | s_t) / \pi_{\text{old}}(a_t | s_t)$$

PPO 优势	PPO 成本/风险
成熟、通用；可建模逐 token value；可处理 dense/process reward	Actor/Reference/RM/Critic 多模型协调；critic bias；显存大；长 CoT value learning 难；hyperparameter 敏感

12. RLOO / ReMax : 回到 REINFORCE 的简化路线

2023-2024 的研究重新审视了“LLM RL 是否真的需要 critic”。ReMax 利用 LLM RL 的 deterministic transition、fast simulation、trajectory-level reward 等特性，去掉 value model；RLOO 则用同 prompt 的其他 samples 作为 leave-one-out baseline。[7][8]

关键启发

“不需要 critic”不是 GRPO 独有思想。RLOO、ReMax 与 GRPO 都在做一件更一般的事：用可廉价获得的 sample-level baseline 替代 learned value function。区别在于 baseline 的统计形式、是否 group normalize、是否 token-level ratio/clipping。

13. PPO vs GRPO : 架构与归纳偏置

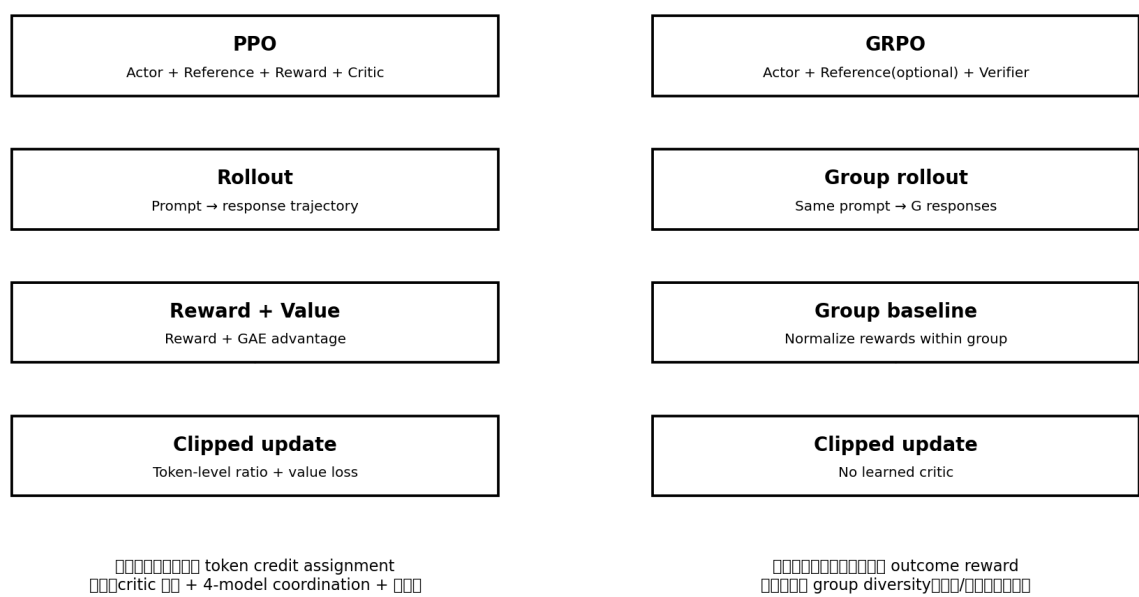


图 3 PPO 与 GRPO 的核心架构差异

维度	PPO	GRPO
优势来源	critic + GAE	group-relative reward
模型副本	通常 Actor+Ref+RM+Critic	Actor+Ref(optional)+Verifier/RM
适合 reward	dense / learned / sparse 均可	同 prompt 可比较的 outcome reward
失败条件	critic 不准、value collapse	group 全同、全对/全错、奖励尺度问题
长 CoT	value 估计难	更自然但易 length/entropy bias
工程复杂度	高	相对低

14. GRPO : 为什么它适合 reasoning RL

DeepSeekMath 首次系统引入 GRPO：针对每个问题采样一组 responses，以 group reward 的均值/标准差构造 advantage，从而不训练一个与 policy 同尺寸的 critic。[4] DeepSeek-R1 延续这一框架并把其用于大规模 reasoning RL。[5]

典型 advantage：

$$A_i = (r_i - \text{mean}(r_1 \dots r_G)) / \text{std}(r_1 \dots r_G)$$

它隐含了一个强归纳偏置：**评价不是“这个答案绝对值是多少”，而是“在同一题当前 policy 的候选里，它相对好多少”**。这特别适合 exact verifier，但也意味着 group 的组成直接决定梯度。

15. GRPO 的偏差与 Dr. GRPO

2025 的“Understanding R1-Zero-Like Training”对 GRPO 做了关键审视：其分析指出某些 normalization / loss 设计会形成 response-length 相关的 optimization bias，并提出 Dr. GRPO 作为更无偏的变体，同时强调 base model 本身的预训练能力对所谓“自发涌现”有重大影响。[10]

Know-Why

看到“RL 让模型学会 Aha moment”时，必须同时问：① base model 在 RL 前是否已经偶尔产生这种行为？② RL 是创造新策略，还是提高已有策略概率？③ evaluation 是否只看 pass@1，忽略 pass@K 与 reasoning diversity？

16. DAPO：把“可复现 long-CoT RL”拆成四个稳定化部件

DAPO 在 Qwen2.5-32B 上从 naive GRPO 的低效/不稳定现象出发，提出四个关键机制：Clip-Higher、Dynamic Sampling、Token-Level Policy Gradient Loss、Overlong Reward Shaping，并基于 verl 开源完整系统。[6]

机制	解决的问题	核心逻辑
Clip-Higher	entropy collapse / 低概率 token 难上升	上调 positive update 的 upper clip，给探索 token 更多空间
Dynamic Sampling	全对/全错 group → 0 advantage	过采样并过滤 degenerate groups
Token-Level PG Loss	sample-level reduction 对长 CoT 权重失真	按 token 聚合，避免长序列被稀释
Overlong Reward Shaping	截断样本被粗暴惩罚造成 reward noise	mask/soft penalty 区分“长但合理”与失控超长

DAPO 的公开训练设置还提供了一个很实用的量级参照：prompt batch 512、每 prompt 16 个 responses、最大生成 20,480 tokens，并使用 $\epsilon_{\text{low}}=0.2 / \epsilon_{\text{high}}=0.28$ 。[6] 这些不是通用最优值，但说明 long-CoT RL 的 rollout 计算量远大于常规 SFT。

17. Entropy Collapse：RL 为什么越训越“只会一种解法”

当 reward 持续奖励一小类策略、clipping 又更容易强化高概率 token，policy entropy 会下降，group 内答案趋同。此时虽然训练 reward 可能继续上升，探索空间却在收缩。DAPO 将此作为 naive GRPO 的核心问题之一，并用 Clip-Higher 增强低概率 token 的上升空间。[6]

监控指标	异常表现	可能措施
Token entropy	持续快速下降	提高探索、调整 clip、temperature、prompt frontier
Group uniqueness	同题 responses 高度重复	增大 sampling diversity / group policy
Pass@K vs pass@1	pass@1 上升但 pass@K 下降	检查策略坍塌与 mode concentration
Reasoning pattern diversity	解法模板单一	增加 diverse prompts / process constraints / self-play

18. Length Bias 与 Long-CoT 训练

Reasoning RL 很容易把“更长”误当成“更好”：更长轨迹有更多机会碰到正确答案，也可能在 loss reduction、reward shaping、truncation 上获得系统性偏差。Dr. GRPO 指出 GRPO 可能产生 length bias；DAPO 也观察到 sample-level loss 与过长截断策略会导致不健康的 response length 演化。[6][10]

工程原则

不要把 length 当一个最终 cosmetic metric。它同时影响：rollout latency、batch straggler、token-level gradient weighting、truncation noise、reward hacking 和产品推理成本。Long-CoT RL 必须把 length 纳入训练信号与系统调度共同管理。

19. Sparse Reward 与 Credit Assignment

Outcome-only RLVR 往往只有 EOS 时 0/1 reward。对 5k-20k token 的 CoT，这意味着一条轨迹里“哪些 token 导致成功”无法直接知道。GRPO 把同一个 sequence-level advantage 广播到整个 response，简单但粒度度；PPO/VAPO 等 value-based 路线尝试用 value function 做更细 credit assignment，但 value bias 又成为新问题。[11]

路线	优点	问题
Outcome reward broadcast	简单、verifier 可靠	错误中间步骤也可能被整体奖励
Process Reward Model	更密集的步骤反馈	PRM 本身需训练且可能错误
Value model / GAE	理论上细粒度	long-CoT value bias + sparse reward 难学
Environment milestones	Agent 任务更自然	需要可观测、可设计的中间状态

20. VAPO : value-based reasoning RL 为什么还没有消失

GRPO 去 critic 并不意味着 value model 路线已经被淘汰。VAPO 在 long-CoT reasoning 中重新设计 value-based PPO，明确指出三个障碍：value model bias、heterogeneous sequence lengths、sparse reward，并通过一组机制提高稳定性。[11]

更一般的认识

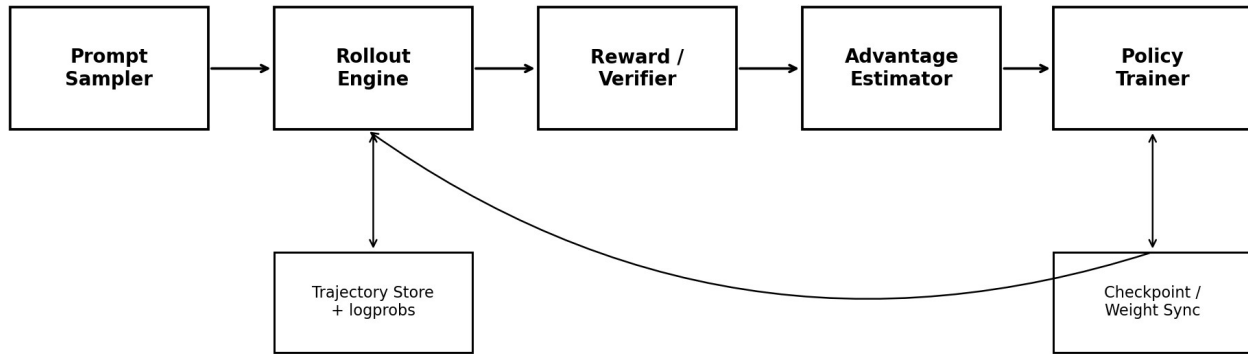
“PPO vs GRPO”不是终局二选一。真正问题是：你的任务是否需要 token/process-level credit？如果 outcome verifier 足够强、group sampling 便宜，critic-free 更有吸引力；如果长轨迹中间决策非常重要，value/process signal 仍可能带来不可替代的信息。

21. Reward Hacking / Verifier Gaming : 最危险的成功

RL 最危险的失败不是 loss 发散，而是训练“成功”地优化了错误目标。Reward Model Overoptimization、RLVR verifier gaming 都说明：policy 会寻找评分函数的边界和漏洞。[3][17]

Hacking 类型	例子	防御
Style hacking	更长、更自信被 judge 偏爱	长度归一、blind judge、多维 rubric
Parser hacking	输出特殊格式骗 exact matcher	canonical parser + adversarial tests
Unit-test hacking	硬编码 hidden-visible cases	更多 hidden tests / metamorphic tests
Extensional shortcut	满足实例而非抽象规则	isomorphic / perturbation verification
Tool/environment loophole	利用环境 bug 获得 task success	sandbox、state invariants、independent

22. RL 系统：为什么 Rollout 往往比 Backprop 更难



$$\text{Production bottleneck} = \text{rollout throughput} \times \text{model sync} \times \text{verifier latency} \times \text{policy freshness}$$

图 4 生产级 LLM RL 的数据流：生成、验证、优势估计、训练与权重同步

RLHF/RLVR 不再是“一个模型做一次 forward+backward”。OpenRLHF 指出大模型 RLHF 需要协调 actor、critic、reward、reference；HybridFlow/verl 则把 RLHF 表述成多个分布式 LLM program 之间的数据流，并设计 training↔generation reshard 以减少通信/显存冗余。[12][14]

系统瓶颈	典型现象	工程解法
Generation straggler	batch 被最长 CoT 拖住	continuous batching / async rollout / length-aware scheduling
Weight sync	trainer 更新后 generator 权重陈旧	fast reshard / weight broadcast / versioning
KV cache	长 CoT 占大量显存	prefix sharing、paged KV、并行 rollout
Verifier latency	代码/环境执行慢	verifier farm、并行 sandbox、cache
GPU fragmentation	训练与生成资源峰谷错位	colocation / hybrid engine / elastic workers

23. Synchronous vs Asynchronous RL

维度	Synchronous	Asynchronous
Policy freshness	高	存在 staleness
实现复杂度	较低	高
GPU 利用率	容易被 longest sample 拖累	更高
稳定性	通常更稳	需 off-policy/staleness correction
代表	DeepSeek/DAPO 类 batch loop	AReaL 等

AReaL 将 rollout workers 与 training workers 完全解耦，并通过 workload balancing 控制 staleness，再配合 staleness-enhanced PPO；论文报告在相同 GPU 数下相对最佳同步系统最高 $2.57\times$ training speedup。[13]

24. Training/Generation Reshard : LLM RL 的独特系统成本

SFT 训练时模型按 optimizer-friendly 方式切分；rollout 时又希望按 inference-friendly tensor/pipeline/data parallel 布局运行。HybridFlow/verl 的 3D-HybridEngine 专门解决 actor 在 training 与 generation 两种模式间的重分片，并报告相对基线 $1.53\times$ 到 $20.57\times$ 的吞吐提升范围。[12]

Know-How

做 RL 系统 capacity planning 时，不要只估“训练 FLOPs”。至少分开算：prompt prefill、decode tokens、verifier execution、old/new logprob recomputation、policy backward、reference/critic forward、weight sync 与 idle/straggler。

25. 核心 KPI : 不要只看 Reward Curve

KPI 层	建议指标
Task	pass@1、pass@K、task success、unseen eval
Reward	mean/std、proxy-vs-independent gap、reward saturation
Policy	KL、entropy、clip fraction、ratio distribution
Experience	effective group ratio、positive/negative balance、unique trajectories
Length	mean/p95/max、truncation rate、overlong fraction
Systems	rollout tok/s、GPU util、trainer idle、weight-sync time、verifier latency
Cost	successful improvement / GPU-hour、verified positive / 1M tokens

Tulu 3 的 RLVR recipe 特别值得借鉴的一点，是把 dev / unseen 多任务 evaluation 与 decontamination 一起纳入 post-training recipe，而不是仅根据 RL reward 选 checkpoint。[15]

26. Production Recipe A : 经典 RM + PPO RLHF

步骤	做法	监控
1	SFT policy + frozen reference	behavior regression
2	Preference data $\rightarrow$ RM	RM pair accuracy / calibration
3	On-policy rollout	length / KL / reward distribution
4	Reward = RM score - β KL	gold/human spot check
5	PPO + critic/GAE	clip frac / value loss / advantage stats
6	Independent human eval	防止 RM overoptimization

适用

开放式帮助性/风格/安全偏好无法被 exact verifier 表达时。

27. Production Recipe B : GRPO / RLVR Reasoning

步骤	做法
1	准备可验证 prompt + canonical answer / unit tests
2	每 prompt 采样 G 个 responses

3	Verifier 计算 outcome reward
4	过滤 degenerate groups 或动态重采样
5	Group-relative advantage
6	Clipped policy update; 按目标选择是否加 KL
7	监控 entropy / length / pass@K / verifier hacks
8	用独立 verifier + unseen benchmark 选 checkpoint

28. Production Recipe C : Coding / Agent RL

Coding/Agent RL 的动作空间不再只有 token，而是包含 tool calls、文件编辑、shell/test、浏览器/环境状态变化。reward 也不应只有最终“成功/失败”，而需要状态不变量、测试、成本与安全约束。

阶段	信号
Repository understanding	是否读取了必要文件；检索召回
Patch	compile / lint / patch validity
Test	unit / integration / hidden tests
Recovery	失败后是否诊断并采取新动作
Final	task success + no-regression
Cost	tool calls、tokens、wall time、environment resets

关键差异

Agent RL 的核心难点是 horizon 与 environment。单纯把最终 reward 广播给整条轨迹会产生严重 credit assignment 问题，因此 milestone verifier、process reward、trajectory critic、hierarchical action abstraction 都更重要。

29. 20 类常见失败模式

#	症状	根因	修复方向
1	Reward ↑，真实能力 ↓	proxy overoptimization	独立 eval / human / gold verifier
2	Group 全对	0 advantage	dynamic sampling / harder prompts
3	Group 全错	0 advantage	curriculum / easier frontier / larger G
4	Entropy collapse	探索消失	clip-higher / sampling diversity
5	Length explosion	更长被间接奖励	length-aware loss/reward
6	Truncation noise	合理长推理被判负	overlong masking / soft penalty
7	Critic bias	错误 advantage	critic-free / value calibration
8	Reward noise	错误梯度	verifier audit / ensemble
9	Verifier gaming	钻规则漏洞	hidden/perturbation verifier
10	KL 太强	policy 学不动	降低 β / objective-specific reference
11	KL 太弱	policy drift	regression eval / adaptive KL
12	Stale rollout	off-policy bias	version gate / IS correction
13	Rollout 太贵	GPU 大量用于 decode	batching / async / experience reuse
14	Long-tail straggler	trainer 等最长样本	length scheduling / async
15	Mode concentration	pass@K 下降	diversity metrics / exploration
16	Reward saturation	全样本同分	harder task / richer verifier
17	Prompt mix 漂移	只在数学变强	multi-task curriculum / eval

18	Benchmark contamination	假提升	strict decontamination
19	Checkpoint 选错	最高 reward 非最佳 model	behavior + unseen eval
20	System metrics 缺失	无法定位训练崩点	trajectory lineage + full telemetry

30. L0-L4 RL Factory 成熟度模型

级别	特征	判定
L0 Script	单机/小规模 PPO/GRPO, 手工 reward	能跑, 但不可追溯
L1 Reproducible	固定 dataset/verifier/config, 完整 metrics	同 seed/recipe 可稳定重现
L2 Signal-aware	reward decomposition、independent eval、dynamic sampling	知道梯度来自哪里
L3 System-aware	rollout/training 解耦、staleness、reshard、cost accounting	能把大规模 RL 训稳训快
L4 Adaptive Experience Controller	failure mining + dynamic task frontier + verifier evolution + experience reuse	系统持续选择 “下一批最值得学的经验”

31. 最终 Know-Why / Know-How 总表

主题	Know-Why	Know-How
RL	在线数据能让 policy 搜索训练集未显式给出的行为	设计 rollout→reward→advantage→update 闭环
PPO	限制 policy step, 支持多轮 minibatch 更新	ratio/clip/GAE/value/KL 全链路监控
GRPO	同题多样本提供廉价相对 baseline	保证 group diversity, 处理 degenerate groups
RLVR	可验证 reward 扩展便宜且客观	把 verifier 当安全关键组件做 audit
Dynamic Sampling	全对/全错没有 relative gradient	持续维护 learnable frontier
Long-CoT	推理能力与序列长度强耦合	同时治理 entropy、length、truncation、token weighting
Systems	RL = distributed generation + distributed training	拆开测 rollout、sync、verifier、trainer 利用率
Evaluation	reward 不是最终真相	训练 reward 与 independent eval 分离

32. 可直接复用的 RL Training Contract

```

rl_stage: reasoning_rlvr
base_checkpoint: sft_or_base_model
prompt_distribution:
  domains: [math, code]
  difficulty_policy: learnable_frontier
rollout:
  samples_per_prompt: 16
  temperature: stage_specific
  max_new_tokens: 16384
  policy_freshness_max_lag: 1
reward:
  type: verifiable_outcome
  components: [correctness, format, length]
  verifier_version: v3
advantage:
  estimator: group_relative
  normalize_within_prompt: true

```

```

optimization:
  algorithm: grpo_or_dapo
  clip_low: 0.2
  clip_high: tuned
  kl_mode: task_dependent
filtering:
  remove_all_correct_groups: true
  remove_all_wrong_groups: true
monitor:
  - independent_pass_at_1
  - pass_at_k
  - entropy
  - effective_group_ratio
  - mean_p95_length
  - verifier_hack_rate
  - rollout_gpu_util
checkpoint_gate:
  require_unseen_eval_improvement: true
  require_no_general_regression: true

```

33. 推荐阅读路线

顺序	论文/报告	为什么读
1	PPO (2017)	理解 clipped policy gradient 的原点
2	Learning to Summarize from Human Feedback (2020)	LLM RM+PPO 的经典闭环
3	InstructGPT (2022)	SFT→RM→PPO→KL 的完整 alignment recipe
4	Reward Model Overoptimization (2022)	理解 Goodhart 与 KL
5	DeepSeekMath (2024)	GRPO 的原始 LLM reasoning 版本
6	Back to Basics / RLOO (2024)	理解 critic-free RL 不只 GRPO 一条路
7	Tulu 3 (2024)	开放 RLVR post-training recipe
8	DeepSeek-R1 (2025)	大规模 reasoning RL 与 multi-stage post-training
9	DAPO (2025)	最重要的 long-CoT 稳定化工程细节之一
10	Dr. GRPO (2025)	理解 GRPO bias 与 base-model confound
11	VAPO (2025)	value-based long-CoT 另一条路线
12	HybridFlow/verl + OpenRLHF	理解生产 RL systems
13	AReAL (2025)	理解异步 rollout 与 staleness
14	EAPo (2026)	理解 experience reuse / off-policy frontier
15	LLMs Gaming Verifiers (2026)	理解 RLVR 的 signal integrity 风险

参考文献（以公开原始论文/官方技术报告为主）

- [1] Ouyang et al. Training language models to follow instructions with human feedback. arXiv:2203.02155.
- [2] Schulman et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- [3] Gao, Schulman, Hilton. Scaling Laws for Reward Model Overoptimization. arXiv:2210.10760.
- [4] Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.
- [5] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.
- [6] Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476.

- [7] Li et al. ReMax: A Simple, Effective, and Efficient Reinforcement Learning Method for Aligning Large Language Models. arXiv:2310.10505.
- [8] Ahmadian et al. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. arXiv:2402.14740.
- [9] Huang et al. The N+ Implementation Details of RLHF with PPO. arXiv:2403.17031.
- [10] Liu et al. Understanding R1-Zero-Like Training: A Critical Perspective. arXiv:2503.20783.
- [11] Yue et al. VAPO: Efficient and Reliable Reinforcement Learning for Advanced Reasoning Tasks. arXiv:2504.05118.
- [12] Sheng et al. HybridFlow: A Flexible and Efficient RLHF Framework. arXiv:2409.19256.
- [13] Fu et al. AReal: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. arXiv:2505.24298.
- [14] Hu et al. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. arXiv:2405.11143.
- [15] Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124.
- [16] Lu et al. Experience Augmented Policy Optimization for LLM Reasoning. arXiv:2606.30420.
- [17] Helff et al. LLMs Gaming Verifiers: RLVR can Lead to Reward Hacking. arXiv:2604.15149.
- [18] Stiennon et al. Learning to summarize from human feedback. arXiv:2009.01325.
- [19] Bai et al. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073.

注：文中的“工程建议”是基于上述公开研究进行的综合与推论；不同模型、任务与算力条件下需通过 ablation 重新验证，不应把单篇论文的超参数直接当成普适 recipe。